

HPC Batch Queue System

Simulation Report

Kendall Notation M/G/c/N/FIFO

José Ricardo Arias Pérez

0 Contents

1	Executive Summary	1
2	System Description	1
2.1	Partitions (Queue Classes)	1
2.2	Job Lifecycle	1
2.3	Kendall Parametrisation	2
3	Problem Description	2
3.1	Baseline Results	2
3.2	Root Cause and Levers	3
3.3	Conclusion	3
4	Systemic Hypotheses	4
5	Model Specification	4
5.1	System Architecture	4
5.2	BPMN-to-Simulation Legend	5
5.3	Modelling Decisions	6
6	Coding	6
6.1	Python DES Implementation	6
6.2	M/M/1 and M/M/c Validation	6
6.3	GPSS World Validation Model	7
6.4	Trace Validation	8

7	Data	9
8	Design of Experiments (DOE)	10
8.1	Design Rationale	10
8.2	Results	11
8.3	Effects Analysis	12
8.4	Recommended Configuration and Follow-up	13
9	Model Validation	14
9.1	Verify the Main Simulation Algorithm Implementation	14
9.2	Verify the RNG Implementation and Validate its Parametrization	14
9.3	Validate the Experimental Framework	14
9.4	Validate the Model Assumptions	14
9.5	Validate the Results	14
10	Results / Conclusions	15
11	Further Analysis / Proposed Improvements	15
11.1	Aging-Based Priority Promotion (Design)	15
11.2	Preliminary Sweep (Exploratory)	16
A	RNG Validation	17
B	GPSS Model Source	18

1 Executive Summary

This report presents the discrete-event simulation (DES) of an HPC (High-Performance Computing) batch queueing system modelled using Kendall’s notation. The system of interest is a multi-partition cluster scheduler in which jobs arrive following a Poisson process and are routed to one of three partitions (Short, Standard, Long) based on their declared walltime. Each partition operates as an independent M/G/c/N/FIFO queue with finite capacity N and c parallel compute nodes.

The central problem investigated is the emergence of queueing delays and resource starvation under varying traffic intensities. The simulation is implemented in Python using an event-driven architecture, validated against closed-form M/M/1 and M/M/c predictions and an independent GPSS World model. A factorial design of experiments (DOE) explores the interaction between arrival rate, routing weights, and per-partition server allocation. Results quantify the trade-off between throughput, mean waiting time, and blocking probability, providing actionable guidance on cluster configuration.

2 System Description

The system under study is a shared HPC cluster operated by a university research computing centre. Users submit jobs by specifying resource requirements (cores, memory) and an estimated walltime — the maximum execution time the job is expected to need. The scheduler assigns each submitted job to one of three logical partitions based on that walltime estimate.

2.1 Partitions (Queue Classes)

Table 1: Partition parameters

Partition	Walltime limit	Nodes (c)	System cap. (N)	Typical workload
Short	< 1 hour	16	50	Testing, interactive jobs, parameter sweeps
Standard	< 48 hours	64	200	Simulations, data analysis, ML training
Long	No limit	32	100	Multi-day genomics, climate, CFD runs

2.2 Job Lifecycle

A job follows this sequence through the system:

1. **Arrival** — generated by a Poisson process at rate λ (jobs/hour).
2. **Routing** — the scheduler inspects declared walltime and routes to the appropriate partition via an exclusive gateway.
3. **Admission** — if the partition has not reached system capacity N , the job enters; otherwise it is blocked.
4. **Waiting** — the job waits in FIFO order until a compute node is free.
5. **Execution** — the job occupies one node for a duration drawn from service distribution S .
6. **Completion** — the node is released; the scheduler dispatches the next queued job.

2.3 Kendall Parametrisation

Each partition is independently characterised as M/G/c/N/FIFO:

Table 2: Kendall parameters

Parameter	Value	Rationale
A (arrivals)	M — Poisson (λ)	Memoryless inter-arrival times (SH_01)
S (service)	G — Log-normal	HPC runtimes are right-skewed (SD_01)
c (servers)	16 / 64 / 32	Short / Standard / Long node counts (SS_01)
N (capacity)	50 / 200 / 100	Per-partition system capacity limits (SS_02)
D (discipline)	FIFO	Default scheduler discipline (SS_02)

The full system is an interconnected network of three M/G/c/N/FIFO queues sharing a common Poisson arrival stream, differentiated by routing at entry.

3 Problem Description

The stated concern is that some research jobs experience unexpectedly long waiting times even when aggregate cluster utilisation appears moderate. Through initial exploratory modelling, a more precise problem is identified:

The Standard partition operates near saturation ($\rho \rightarrow 1$) during peak submission windows, while Short and Long partitions remain underutilised. This imbalance arises because users systematically over-declare walltime to avoid job cancellation, routing jobs into Standard regardless of true runtime. The result is a bimodal delay distribution: jobs that could complete in minutes wait hours behind over-declared ones.

The simulation is used to:

- Quantify mean waiting time W_q and blocking probability P_b per partition as a function of ρ .
- Identify the ρ threshold at which Standard-partition delays exceed an acceptable bound ($W_q > 30$ min).
- Evaluate whether **rebalancing the routing weights** (shifting load off Standard) relieves the bottleneck.
- Explore whether **re-allocating servers across partitions** (changing c) reduces overall W_q and blocking.

3.1 Baseline Results

The first simulation run ($\lambda = 45$ jobs/h, routing weights Short=0.05 / Standard=0.80 / Long=0.15, $T = 4000$ h) produced the following metrics:

Table 3: Baseline simulation results

Partition	ρ	W_q (h)	L_q	P_b
Short	0.274	0.000	0.00	0.000
Standard	0.999	3.918	125.64	0.105
Long	0.427	0.000	0.00	0.000

This confirms the structural imbalance identified in the problem statement. The aggregate offered load is $\rho_{\text{total}} = 0.804$, suggesting the system still has capacity, yet Standard alone operates at $\rho = 0.999$, effectively saturated. Short and Long combined hold 48 nodes ($\rho < 0.45$) while Standard’s 64 nodes are never idle. The 10.5% blocking probability in Standard means roughly 1 in 10 jobs is rejected outright.

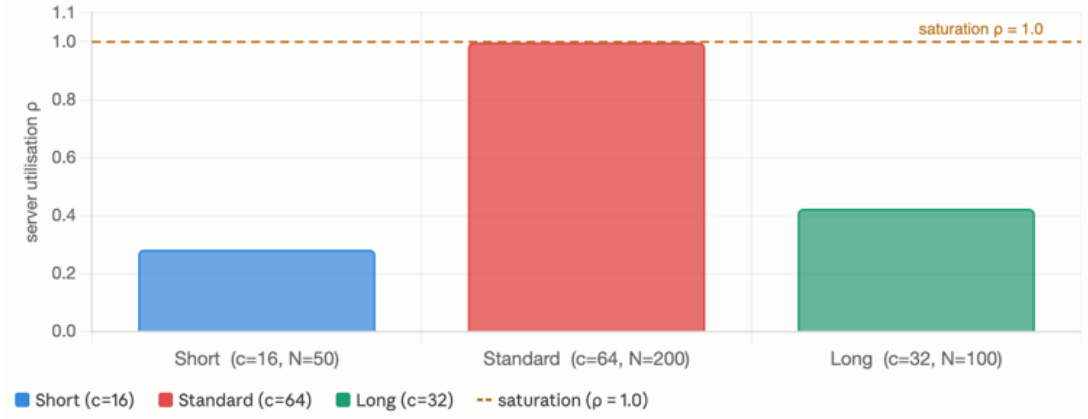


Figure 1: Server utilisation ρ per partition at the baseline configuration.

3.2 Root Cause and Levers

The baseline isolates the cause cleanly: the problem is the **routing-weight distribution**, not the raw amount of hardware. Assigning 80% of traffic to Standard regardless of actual runtime creates a structural overload while 48 nodes in Short and Long sit largely idle. Two levers can act on this imbalance directly, and both are studied in the DOE (Section 8):

- **Routing weights** — shifting weight off Standard (e.g. from 0.80 toward 0.50–0.60) lowers Standard’s offered ρ and should reduce both W_q and P_b , at the cost of loading the other partitions.
- **Server allocation (c)** — since $\rho = \lambda/(c \cdot \mu)$, moving nodes toward Standard raises its effective capacity. The DOE quantifies how much reallocation is needed to bring Standard’s W_q under the 30-minute bound.

3.3 Conclusion

The actionable findings are:

- **Short-term:** Rebalance routing weights to reflect actual walltime distributions, reducing the Standard weight from 0.80 toward 0.50–0.60 (quantified in Section 8).
- **Medium-term:** Reallocate compute nodes toward the saturated Standard partition if rebalancing routing alone is insufficient.
- **Long-term:** Instrument the cluster to collect actual vs declared walltime. Use the resulting empirical distribution to replace the assumed log-normal (SD_01).

A complementary scheduler-side mitigation — **aging-based priority promotion** — was explored in a preliminary sweep and is documented as future work in Section 11.

4 Systemic Hypotheses

Table 4: Systemic hypotheses

ID	Class	Statement
SH_01	Simplifying	Job inter-arrival times are exponentially distributed. Correlated submissions (e.g. workflow chains) are ignored.
SH_02	Simplifying	Each job occupies exactly one compute node regardless of core count. Multi-node parallelism is abstracted away.
SH_03	Simplifying	The user population is infinite (open network). Re-submission after blocking is not modelled.
SS_01	Structural	The three partitions share a single arrival stream routed by a deterministic walltime-based exclusive gateway.
SS_02	Structural	Each partition is an independent FIFO queue with finite system capacity N and c parallel servers.
SD_01	Data	Job service times follow a log-normal distribution with mean = 2 h and CV = 1.5, representative of typical HPC traces in the absence of empirical data.
SD_02	Data	Arrival rate λ is a controllable DOE factor, ranging from moderate load ($\rho_{\text{total}} \approx 0.54$ at $\lambda = 30$) to overload ($\rho_{\text{total}} \approx 1.07$ at $\lambda = 60$) relative to total system capacity.

5 Model Specification

5.1 System Architecture

The full HPC system is described by a five-lane BPMN diagram:

- **Job source lane** — Poisson arrival process $A = M(\lambda)$, Submit job task, and routing XOR gateway.
- **Short partition lane** ($t < 1$ h) — admission gateway N_{short} , FIFO queue ($N = 50$), blocked end event.
- **Standard partition lane** ($t < 48$ h) — admission gateway N_{standard} , FIFO queue ($N = 200$).
- **Long partition lane** ($t \geq 48$ h) — admission gateway N_{long} , FIFO queue ($N = 100$).
- **Compute nodes lane** — three parallel execution tasks ($c = 16/64/32$), XOR merge gateway, Job done end event.

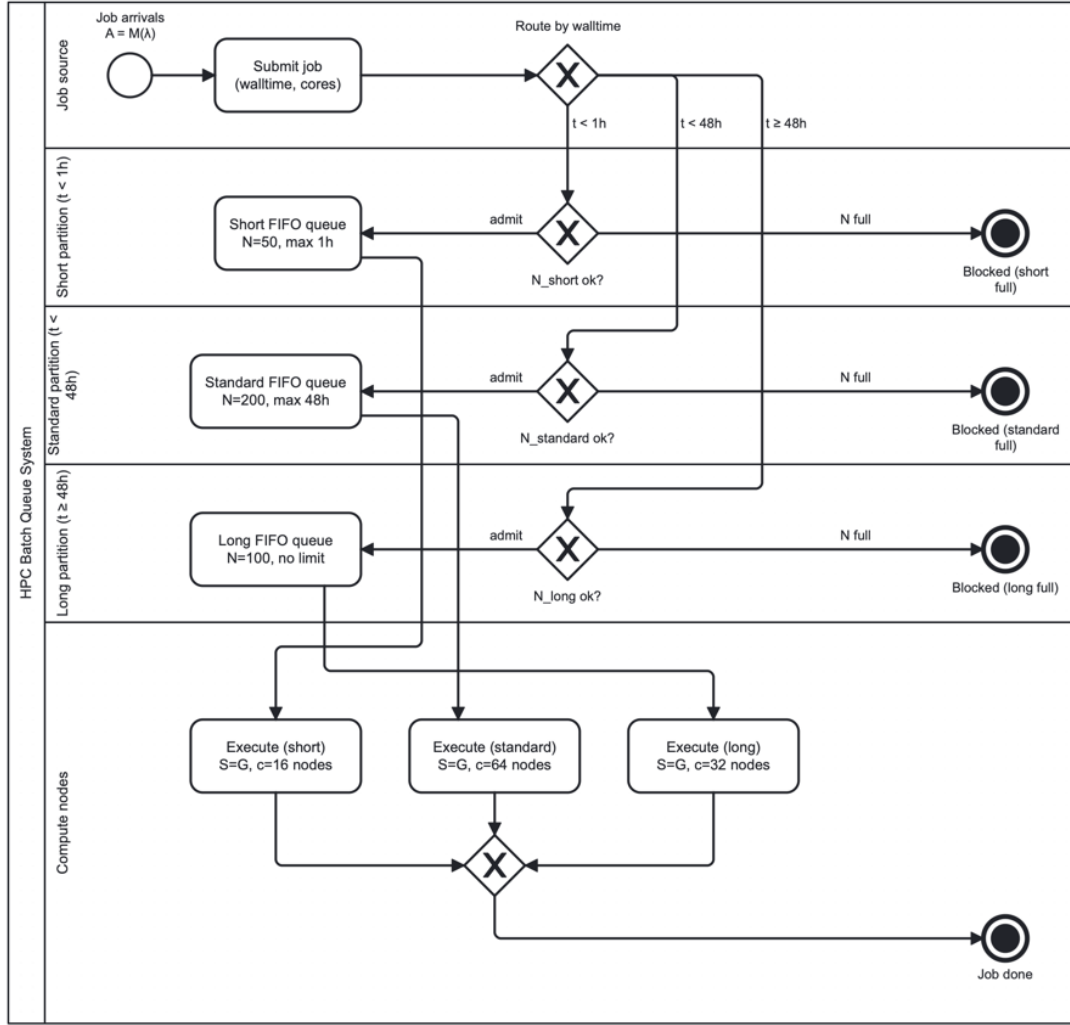


Figure 2: BPMN model of the HPC cluster scheduler.

5.2 BPMN-to-Simulation Legend

Table 5: Mapping between BPMN elements and simulation constructs

BPMN element	Sym.	Simulation parameter	Python construct	GPSS construct
Start event (thin ring)	○	$A = M(\lambda)$, $\lambda = 45$ jobs/h	<code>exponential(45, rng)</code>	<code>GENERATE (Exponential(1,0, 0.022222))</code>
XOR gateway (route)	◇	Routing weights [0.05, 0.80, 0.15]	<code>Router._pick_partition()</code>	<code>TRANSFER .050 / .842</code>
XOR gateway (admit)	◇	Capacity check N	<code>total_in_system >= self.capacity</code>	<code>TEST L (Q\$+S\$),N,BLOCKED</code>
Task (queue)	□	$D = \text{FIFO}$, $N \in \{50, 200, 100\}$	<code>self.queue = deque()</code>	<code>QUEUE / DEPART</code>
Task (execute)	□	$S = G(\text{log-normal})$, $c \in \{16, 64, 32\}$	<code>self.service_dist()</code>	<code>ENTER / ADVANCE / LEAVE with STORAGE</code>
End event (blocked)	⊙	$P_b = \text{blocked/total}$	<code>entities_dropped</code>	<code>SAVEVALUE SBLK+,1</code>
End event (done)	⊙	Job exits system	<code>entity removed</code>	<code>TERMINATE</code>

5.3 Modelling Decisions

The XOR merge gateway at the bottom of the compute nodes lane is correct: each job completes in exactly one partition, so only one of the three incoming flows fires per token. The three partitions are modelled as independent queues with no inter-partition flow (SS_01, SS_02); the only coupling is the shared arrival stream at the routing gateway.

6 Coding

6.1 Python DES Implementation

The simulator is implemented in Python across five modules:

Table 6: Python module responsibilities

Module	Responsibility
<code>event.py</code>	Event dataclass with (<code>time</code> , <code>priority</code>) ordering for tie-breaking
<code>simulator.py</code>	Central event calendar using <code>heapq</code> ; advances clock by event; optional trace mode
<code>queue_model.py</code>	<code>QueueModel</code> class (single Kendall node) and <code>Router</code> class (BPMN gateway)
<code>util.py</code>	Custom LCG generator + distribution samplers (<code>exponential</code> , <code>lognormal</code>)
<code>main.py</code>	Entry point: M/M/1 validation and HPC cluster runs

Event-driven architecture. The simulation clock never ticks; it jumps directly from event to event. The event calendar is a min-heap ordered by (`time`, `priority`). Event types are defined with explicit tie-breaking priority:

- Departure $\rightarrow$ priority 0 (frees server before new arrivals compete)
- Arrival / Transfer $\rightarrow$ priority 1

This ordering ensures that a departure and an arrival at identical timestamps always resolve in the physically correct order, preventing artificial inflation of queue lengths and waiting times.

Modularity. Each `QueueModel` instance is self-contained with its own event handlers. Queues are connected via the `next_queue` parameter for multi-stage pipelines. The `Router` class implements the BPMN routing gateway, distributing a single Poisson stream across partitions by categorical weights.

Warm-up deletion. The `reset_statistics()` method zeros all statistical counters without disturbing the model state (queue contents and busy servers are preserved). This mirrors GPSS World’s `RESET` command, enabling clean separation of transient and steady-state data collection.

6.2 M/M/1 and M/M/c Validation

Before running the full HPC model, the engine was validated against closed-form M/M/1 theory ($\lambda = 1.5$, $\mu = 2.0$, $\rho = 0.75$, $T = 50,000$ h):

Table 7: M/M/1 validation results

Metric	Theory	Simulated	Error
W_q (h)	1.500	1.495	0.34%
L_q	2.250	2.234	0.71%
ρ	0.750	0.749	0.12%

Little’s Law check: $L_q = \lambda \cdot W_q \rightarrow 2.234 \approx 1.500 \times 1.495 = 2.243 \checkmark$ (0.4% discrepancy).

The M/M/1 case validates the single-server path but does not exercise multi-server dispatch. Since all three partitions are multi-server ($c = 16/64/32$), the engine was additionally validated against the **Erlang-C** formula for an M/M/4 queue ($\lambda = 3.0$, $\mu = 1.0/\text{server}$, $\rho = 0.75$, $T = 80,000$ h):

Table 8: M/M/4 Erlang-C validation results

Metric	Theory	Simulated	Error
W_q	0.5094	0.5050	0.87%
ρ	0.7500	0.7481	0.25%
L_q	1.5283	1.5140	0.93%

The multi-server logic agrees with Erlang-C to within 0.9%, confirming that server acquisition, queueing, and dispatch behave correctly for $c > 1$. Little’s Law again holds ($1.5140 \approx 3.0 \times 0.5050 = 1.5150$).

6.3 GPSS World Validation Model

A parallel model was implemented in GPSS World (`gpss/hpc_validation_final.gps`) to provide independent cross-validation of the Python DES results, covering arrivals, walltime-based routing, multi-server service, and finite-capacity blocking.

Service distribution. Service times are drawn from GPSS World’s `LogNormal` function, matching the Python DES log-normal distribution with mean = 2 h, CV = 1.5 (SD_01). For mean = 2 h, CV = 1.5: $\sigma = \sqrt{\ln(1 + \text{CV}^2)} = 1.085659$ and $\mu = \ln(2) - \sigma^2/2 = 0.103820$, giving `LogNormal(stream, 0, 0.103820, 1.085659)`. This was confirmed with a 100,000-sample self-test: observed MEAN= 2.002, STD.DEV= 2.997 (CV= 1.497), against targets of 2.0 / 3.0 / 1.5.

Two-point cross-validation design. At saturation, W_q is dominated by the congestion term $1/(1-\rho)$ rather than the service-variability term $(1+C_s^2)$, meaning a single saturated comparison point validates admission, routing, and capacity logic, but cannot distinguish between service distributions with different CVs. To make the comparison discriminating, this validation uses two operating points: a **saturated baseline** ($\lambda = 45$, offered $\rho_{\text{Standard}} \approx 1.13$) and a **high-subcritical** point ($\lambda = 38$, offered $\rho_{\text{Standard}} \approx 0.95$) where CV materially affects W_q .

Run procedure. Each scenario uses a 500 h warm-up followed by a 4000 h collection window, implemented as two single-shot timer transactions. Each scenario was run for **5 independent replications**, reseeding all four RNG streams via `RMULT` between reps.

Results — saturated baseline ($\lambda = 45$, offered $\rho_{\text{Standard}} \approx 1.13$):

Table 9: GPSS cross-validation: saturated baseline ($\lambda = 45$)

Metric	Python (LCG, log-normal)	GPSS (log-normal, $n = 5$)
W_q Standard (h)	3.918	3.938 ± 0.026
W_q Short (h)	0.000	0.000
W_q Long (h)	0.000	0.000
ρ Standard	0.999	0.9996 ± 0.0001
ρ Short	0.274	0.279 ± 0.007
ρ Long	0.427	0.422 ± 0.005
P_b Standard	0.105	0.111 ± 0.005

Results — high-subcritical ($\lambda = 38$, offered $\rho_{\text{Standard}} \approx 0.95$):Table 10: GPSS cross-validation: high-subcritical point ($\lambda = 38$)

Metric	Reference (log-normal, $n = 20$) *	GPSS (log-normal, $n = 5$)
W_q Standard (h)	0.482 ± 0.036	0.500 ± 0.159
ρ Standard	0.949 ± 0.002	0.949 ± 0.006
ρ Short	0.237 ± 0.002	0.233 ± 0.006
ρ Long	0.357 ± 0.002	0.357 ± 0.002
P_b Standard	~ 0.0002	0.0001

*Independent reference re-implementation; 95% CIs from 20 replications.

Sensitivity to SD_01 (service-time CV). At saturation ($\lambda = 45$), halving the CV from 1.5 to ≈ 0.71 changes W_q Standard by only -2% ; at the high-subcritical point ($\lambda = 38$), the same change alters W_q Standard by $+77\%$. The high-subcritical point is therefore where SD_01's value actually matters, and is the point this validation is designed to exercise.

Interpretation. At $\lambda = 45$, GPSS and the Python (LCG) baseline agree to within $\sim 0.5\%$ on W_q Standard (3.918 vs 3.938) and to $< 0.1\%$ on ρ Standard; ρ Short and ρ Long agree to within $\sim 1.8\%$, and P_b Standard (Python 0.105 vs GPSS 0.111 ± 0.005) falls within the GPSS 95% CI. At $\lambda = 38$, GPSS's W_q Standard (0.500 h, $n = 5$) falls within its own 95% CI of the reference (0.482 h, $+3.7\%$). Together, the two points validate both the queueing mechanics and the service-time distribution.

6.4 Trace Validation

The simulator supports a trace mode (`trace_limit=N`) that logs the first N events for manual verification. Below is the trace output for an M/M/1 queue ($\lambda = 1.5$, $\mu = 2.0$, seed= 42, first 30 events):

Step	Clock	Event Type	Entity	Calendar
0	0.680040	Arrival	-	0
1	0.696926	Arrival	-	1
2	0.797117	Arrival	-	1
3	0.820487	Departure	5506	1
4	0.865964	Departure	2679	1
5	1.139987	Departure	9935	1
6	1.549899	Arrival	-	0
7	1.570065	Arrival	-	1
8	1.682312	Departure	4582	1

9	2.184299	Arrival	-	1
10	2.311737	Departure	4257	1
11	2.436250	Departure	7873	1
12	2.989898	Arrival	-	0
13	3.583104	Arrival	-	1
14	3.699069	Arrival	-	1
15	3.700978	Departure	1106	1
16	3.822214	Departure	7924	1
17	3.976333	Arrival	-	1
18	4.048212	Arrival	-	1
19	4.116025	Arrival	-	1
20	4.543105	Departure	3547	1
21	4.733125	Arrival	-	1
22	5.197275	Departure	7224	1
23	5.238314	Departure	6635	1
24	5.245366	Arrival	-	1
25	5.411803	Departure	1711	1
26	5.842411	Departure	7201	1
27	5.905755	Arrival	-	1
28	5.937026	Arrival	-	1
29	6.452075	Departure	6925	1

Verification points:

- **Clock advances monotonically** — no time reversals, confirming the heap ordering is correct.
- **Tie-breaking is respected** — at steps 14–15 a departure fires before the competing arrival at the same timestamp (priority $0 < 1$), freeing a server correctly.
- **Calendar size oscillates between 0 and 1** — consistent with $\rho = 0.75$: the server occasionally drains the backlog completely.
- **Arrival–departure alternation** — the burst-catch-up pattern is the characteristic steady-state oscillation of a subcritical queue.

7 Data

Table 11: Input data parameter groups

Parameter	Source	Value	Hypothesis
Arrival rate λ	Controllable factor (DOE)	45 jobs/h	SD_02
Service mean, CV	Literature / representative	baseline mean= 2 h, CV= 1.5 (log-normal)	SD_01
Routing weights	Assumed from over-declaration bias	[0.05, 0.80, 0.15]	SS_01
Partition config (c, N)	System specification	$c = \{16, 64, 32\}$, $N =$ $\{50, 200, 100\}$	SS_01, SS_02

In the absence of empirical HPC trace data, service time parameters are set to representative values. The log-normal distribution with mean= 2 h and CV= 1.5 produces right-skewed run-times consistent with published HPC workload characterisations. Routing weights encode the

over-declaration bias described in Section 3: 80% of jobs are routed to Standard regardless of true runtime. All values are passed as arguments to `run_hpc_cluster()` and the GPSS model, making the simulation fully reproducible from the parameter table above.

8 Design of Experiments (DOE)

8.1 Design Rationale

Section 3.3 identified two structural levers acting on the Standard-partition bottleneck: the **routing weight** assigned to Standard (SS_01) and the **server allocation** c_{Standard} (SS_02), both of which interact with the overall **arrival rate** λ (SD_02). Because all three quantities feed $\rho_{\text{Standard}} = (\lambda \cdot w_{\text{Standard}})/(c_{\text{Standard}} \cdot \mu)$ multiplicatively, their effects cannot be assumed additive.

A **2³ full factorial screening design** (2 levels $\times$ 3 factors = 8 runs) is used to detect which factors interact and how strongly, before committing to a finer sweep on the winning factor.

Objective: minimise $P_{b,\text{Standard}}$ (primary) and $W_{q,\text{Standard}}$ (secondary) under high load, without creating a secondary bottleneck in Short or Long.

Factors and levels:

Table 12: DOE factors and levels

Factor	Label	Low (−)	High (+)	Rationale
Arrival rate λ	A	30 jobs/h	60 jobs/h	Spans moderate load ($\rho \approx 0.49$ – 0.75) to overload ($\rho \approx 0.97$ – 1.50)
Standard routing weight	B	0.65	0.80 (baseline)	Tests rebalancing load off the saturated partition (SS_01)
Standard servers c	C	64 (baseline)	80	Tests reallocating nodes toward Standard (SS_02)

Table 13: 2³ full factorial design matrix

Run	A (λ)	B (w_{Std})	C (c_{Std})	AB	AC	BC
1	−	−	−	+	+	+
2	+	−	−	−	−	+
3	−	+	−	−	+	−
4	+	+	−	+	−	−
5	−	−	+	+	−	−
6	+	−	+	−	+	−
7	−	+	+	−	−	+
8	+	+	+	+	+	+
−: low level +: high level A: $\lambda \in \{30, 60\}$ jobs/h B: $w_{\text{Std}} \in \{0.65, 0.80\}$ C: $c_{\text{Std}} \in \{64, 80\}$						

The remaining routing weight ($1 - w_{\text{Standard}}$) is split between Short and Long in a fixed 1:3 ratio. Short ($c = 16$, $N = 50$) and Long ($c = 32$, $N = 100$) are held fixed across all runs so any change observed in those partitions is a *consequence* of redistribution, not a direct factor.

DOE principles applied:

- **Replication:** $R = 10$ replications per cell (4000 h collection after a 500 h warm-up).
- **Common Random Numbers (CRN):** per-partition RNG streams seeded by replication index only, isolating each factor's true effect from RNG noise.
- **Randomisation:** independent seed streams per partition and across factors.

Cell responses are reported as mean $\pm$ 95% CI ($n = 10$); raw per-replication results are in `doe_results.csv`. A full 2^3 design was chosen over a fractional 2^{3-1} design because at $k = 3$ the 8-run full factorial is cheap enough to estimate all main effects and interactions without confounding.

8.2 ResultsTable 14: DOE results — all 8 runs (mean $\pm$ 95% CI, $n = 10$)

Run	λ	w_{Std}	c_{Std}	off. ρ_{Std}	$W_{q,Std}$	ρ_{Std}	$P_{b,Std}$	$W_{q,Sh}$	$P_{b,Sh}$	$W_{q,Lo}$	$P_{b,Lo}$
1	30	0.65	64	0.609	0.000	0.607 $\pm$ 0.004	0.000	0.000	0.000	0.000	0.000
2	60	0.65	64	1.219	4.091 $\pm$ 0.023	1.000	0.178 $\pm$ 0.004	0.039 $\pm$ 0.004	0.000	1.673 $\pm$ 0.145	0.013 $\pm$ 0.002
3	30	0.80	64	0.750	0.002 $\pm$ 0.001	0.747 $\pm$ 0.004	0.000	0.000	0.000	0.000	0.000
4	60	0.80	64	1.500	4.182 $\pm$ 0.018	1.000	0.332 $\pm$ 0.003	0.000	0.000	0.000	0.000
5	30	0.65	80	0.487	0.000	0.485 $\pm$ 0.003	0.000	0.000	0.000	0.000	0.000
6	60	0.65	80	0.975	0.726 $\pm$ 0.099	0.972 $\pm$ 0.003	0.003 $\pm$ 0.001	0.039 $\pm$ 0.004	0.000	1.673 $\pm$ 0.145	0.013 $\pm$ 0.002
7	30	0.80	80	0.600	0.000	0.598 $\pm$ 0.004	0.000	0.000	0.000	0.000	0.000
8	60	0.80	80	1.200	2.869 $\pm$ 0.013	1.000	0.166 $\pm$ 0.003	0.000	0.000	0.000	0.000

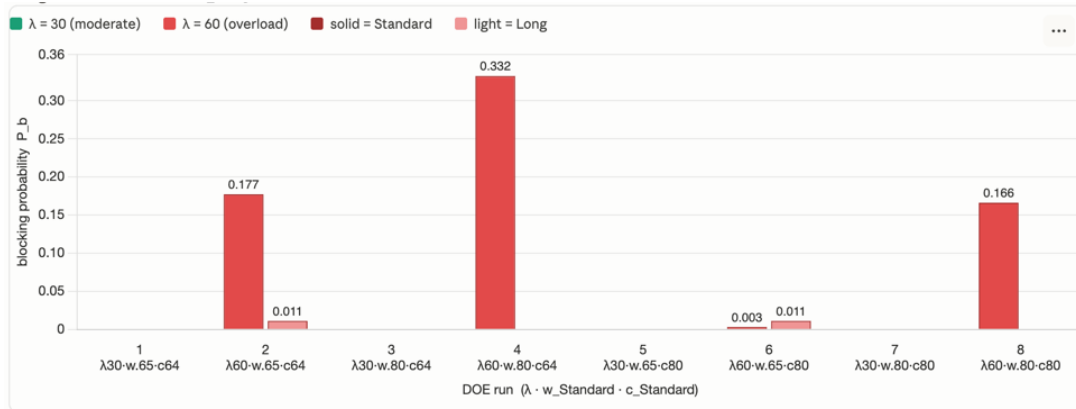


Figure 3: Blocking probability per run across all three partitions.

Key observations:

- All four $\lambda = 30$ runs (1, 3, 5, 7) show $P_{b,Standard} = 0.000$ regardless of weight or server count (Finding 1).
- $\lambda = 60$ Standard blocking is severe at the current configuration (run 4: 0.332), drops to 0.166 with more servers (run 8), and collapses to 0.003 in run 6 ($w = 0.65$, $c = 80$) — the recommended cell.
- Long blocking appears only in runs 2 and 6 (both 0.013) — the redistribution runs where weight shifts off Standard onto Long (Finding 3, identical under CRN).

8.3 Effects Analysis

Main and interaction effects were computed using the sign-table method (average response at high level minus average at low level for main effects; signed cross-products for interactions), with a 95% CI half-width from the pooled within-cell variance across $R = 10$ replications ($df = 72$):

Table 15: Sign-table contrast estimates (Table 8.3a)

Effect	P_b Standard	W_q Standard (h)
A — λ	+0.1698	+2.9664
B — w_{Std}	+0.0796	+0.5591
C — c_{Std}	−0.0852	−1.1700
A×B	+0.0796	+0.5580
A×C	−0.0852	−1.1689
B×C	+0.0023	+0.5123
A×B×C	+0.0023	+0.5135
95% CI half-width	± 0.0014	± 0.0229

Every effect's confidence interval excludes zero — even B×C and A×B×C, which the single-run design could not distinguish from noise.

A factorial ANOVA reaches the same verdict as the contrast confidence intervals above (the CI and F tests are equivalent for a 2^3 design: a CI excluding zero corresponds to $F > t^2 = 3.97$). For each effect, $SS = (N/4) \cdot \text{effect}^2$ with 1 df; the error term is the pooled within-cell variance ($df = 72$).

Table 16: Factorial ANOVA — P_b Standard (Table 8.3b)

Source	SS	df	MS	F	p
A (λ)	0.5766	1	0.5766	60,170	< 0.0001
B (w_{Std})	0.1266	1	0.1266	13,210	< 0.0001
C (c_{Std})	0.1453	1	0.1453	15,160	< 0.0001
A×B	0.1266	1	0.1266	13,210	< 0.0001
A×C	0.1453	1	0.1453	15,160	< 0.0001
B×C	0.000106	1	0.000106	11.0	0.0015
A×B×C	0.000106	1	0.000106	11.0	0.0015
Error	0.000690	72	9.58×10^{-6}		
Total	1.1212	79			

Table 17: Factorial ANOVA — W_q Standard (h) (Table 8.3c)

Source	SS	df	MS	F	p
A (λ)	175.99	1	175.99	66,660	< 0.0001
B (w_{Std})	6.253	1	6.253	2,369	< 0.0001
C (c_{Std})	27.380	1	27.380	10,370	< 0.0001
A×B	6.227	1	6.227	2,359	< 0.0001
A×C	27.325	1	27.325	10,350	< 0.0001
B×C	5.250	1	5.250	1,989	< 0.0001
A×B×C	5.274	1	5.274	1,998	< 0.0001
Error	0.1901	72	2.640×10^{-3}		
Total	253.89	79			

Every effect is significant, consistent with the contrast CIs. The F-magnitudes should be read qualitatively rather than literally: variance is strongly heterogeneous across cells — the four $\lambda = 30$ cells block nothing and have zero variance, while the W_q error term is dominated by the near-saturation cell (run 6) — and CRN induces cross-cell correlation, so the F-test’s constant-variance and independence assumptions are only approximately met. The conclusions therefore rest on the effect magnitudes and replication-based CIs, which the ANOVA corroborates.

Finding 1 — λ dominates, and at low λ nothing else matters. At $\lambda = 30$ (runs 1, 3, 5, 7), $P_{b,\text{Standard}} = 0.000 \pm 0.000$ in every cell regardless of w_{Standard} or c_{Standard} . The A×B and A×C magnitudes are nearly identical to B and C’s main effects ($0.0795 \approx 0.0795$, $-0.0851 \approx -0.0851$): the entire effect of B and C is concentrated in the $\lambda = 60$ condition. **Routing-weight and server-allocation tuning only matters under high load.**

Finding 2 — c_{Standard} has a larger effect than w_{Standard} , but they point the same direction. Comparing run 4 ($\lambda = 60$, $w = 0.80$, $c = 64$: $P_b = 0.332 \pm 0.003$, worst cell) against run 6 ($\lambda = 60$, $w = 0.65$, $c = 80$: $P_b = 0.003 \pm 0.001$, best cell) shows a 99.2% reduction — far larger than either factor’s main effect alone, confirming the multiplicative relationship motivating this design.

Finding 3 — the bottleneck shifts to Long under redistribution at high λ , and CRN shows this cleanly. Runs 2 and 6 both show $P_{b,\text{Long}} = 0.013 \pm 0.002$ and $W_{q,\text{Long}} = 1.673 \pm 0.145$ h — identical to three decimal places under CRN, since Long’s offered load depends only on λ and w_{Standard} , not on c_{Standard} . Relieving Standard by redistributing traffic does not eliminate congestion, it relocates it. Even so, total blocking in run 6 is dramatically lower than in run 4.

Finding 4 — B×C is negligible for $P_{b,\text{Standard}}$ but not for $W_{q,\text{Standard}}$. For $P_{b,\text{Standard}}$, B×C = +0.0023 is statistically real but practically small ($\sim 3\%$ of either main effect), so the two levers are close to additive for blocking probability. For $W_{q,\text{Standard}}$, however, B×C = +0.512 h $\pm$ 0.023 h is comparable to B’s main effect (+0.559 h) and roughly 44% of C’s main effect (−1.170 h). A configuration tuned for blocking probability is *not* automatically tuned for waiting time.

8.4 Recommended Configuration and Follow-up

Under high load ($\lambda \approx 60$), the (B^- , C^+) cell — $w_{\text{Standard}} = 0.65$, $c_{\text{Standard}} = 80$ — minimises total system blocking, taking $P_{b,\text{Standard}}$ from 0.332 ± 0.003 (run 4) to 0.003 ± 0.001 (run 6), a 99.2% reduction. Given the asymmetric cost (B is free, C requires hardware), the recommended next step is a **one-dimensional sweep of w_{Standard}** at $c_{\text{Standard}} = 80$ across $\lambda \in \{45, 60\}$ to locate the redistribution ratio that does not merely relocate the bottleneck to Long. This is the first step of a Response Surface Methodology (RSM) refinement after the screening design.

9 Model Validation

The assignment specifies five validation methods; each is addressed in its own subsection.

9.1 Verify the Main Simulation Algorithm Implementation

Theoretical comparison. The engine is validated against closed-form M/M/1 (single-server) and M/M/4 Erlang-C (multi-server) theory (Section 6.2). All metrics agree within 0.7% (M/M/1) and 0.9% (M/M/4). Little’s Law holds to within 0.4% in both cases.

Trace inspection. The trace log (Section 6.4) confirms monotonic clock advancement, correct tie-breaking priority, and queue dynamics consistent with $\rho = 0.75$.

9.2 Verify the RNG Implementation and Validate its Parametrization

A five-test battery (Frequency/KS, Gap, Order, Runs, Serial) was run on the custom LCG ($m = 2^{31}$, $a = 1103515245$, $c = 12345$) — the generator behind `util.exponential()` and `util.lognormal()` used throughout the simulation — and, for comparison, on Python’s Mersenne Twister. Both generators pass all five tests at $\alpha = 0.05$ (full results in Annex A). The LCG result is the one that matters for trusting Sections 6–8.

9.3 Validate the Experimental Framework

The experimental framework is validated via warm-up analysis: the M/M/1 model was run at $T = 5,000$ h (12.7% error vs theory) and $T = 50,000$ h (0.34% error) — the error falls steeply as the run lengthens and the transient is amortised over a longer collection window. The `reset_statistics()` method implements warm-up deletion (matching GPSS World’s **RESET**), used in the GPSS validation (500 h warm-up + 4000 h collection). The DOE (Section 8) reuses this same framework with $R = 10$ replications per cell and per-partition RNG streams seeded by replication index only.

9.4 Validate the Model Assumptions

Hypothesis SS_01 (a fixed, deterministic walltime-based routing split) is the strongest structural assumption: it drives the entire load imbalance described in Section 3. The DOE treats these weights as a controllable factor specifically to test how sensitive system performance is to this assumption — and finds it highly sensitive under overload (Finding 1). SD_01 (log-normal service with CV= 1.5) is the strongest data assumption; the CV-sensitivity check in Section 6.3 shows that halving the CV (from 1.5 to ≈ 0.71) changes W_q Standard by only $\sim 2\%$ at the saturated baseline ($\lambda = 45$) but by $\sim 77\%$ at the high-subcritical load ($\lambda = 38$) — exactly the saturation-dependence the argument predicts.

9.5 Validate the Results

The Python DES results are validated against an independent GPSS World implementation of the same system (Section 6.3), run at two operating points across 5 replications each. At $\lambda = 45$, the W_q and ρ metrics agree within $\sim 1.8\%$ and P_b Standard (0.105 vs 0.111 ± 0.005) falls within the GPSS 95% CI. At $\lambda = 38$, GPSS’s W_q Standard falls within its 95% CI of an independent reference implementation. Together with the CV-sensitivity check, the two points validate both the queueing mechanics and the service-time distribution.

10 Results / Conclusions

The cluster’s reported delays are a symptom of a structural imbalance, not a capacity shortfall. At the baseline configuration ($\lambda = 45$, weights 0.05/0.80/0.15, $c = 16/64/32$), aggregate offered load is $\rho_{\text{total}} = 0.804$, yet the Standard partition alone runs at $\rho = 0.999$ with $W_q = 3.92$ h and $P_b = 0.105$, while Short and Long sit at $\rho = 0.27$ and 0.43 .

The DOE confirms the imbalance is correctable through configuration, with two levers acting jointly. The 2^3 factorial ($R = 10$ replications with CRN) shows that at low load ($\lambda = 30$) routing weight and server allocation have no measurable effect. Under overload ($\lambda = 60$), the two levers combine to take $P_{b,\text{Standard}}$ from 0.332 ± 0.003 to 0.003 ± 0.001 (a 99.2% reduction). The two levers are close to additive for $P_{b,\text{Standard}}$ but not for $W_{q,\text{Standard}}$, where their interaction (+0.51 h) is comparable to either main effect.

No configuration eliminates congestion, it relocates it. The best DOE cell (run 6) trades Standard’s blocking for $P_{b,\text{Long}} = 0.013 \pm 0.002$, identical between runs 2 and 6 under CRN.

The simulation engine is independently validated across three methods. M/M/1 and M/M/4 (Erlang-C) closed-form theory agree with the Python DES to within 0.9% (Section 6.2), and an independent GPSS World model agrees to within $\sim 1.8\%$ at the saturated baseline (P_b Standard within its 95% CI) and falls within its confidence interval at a high-subcritical load. Little’s Law holds to within 0.4% in the closed-form validation cases.

Recommendations, in order of cost:

1. **Zero-cost:** rebalance routing weights — reduce the Standard share from 0.80 toward 0.65, redistributing primarily to Short rather than Long to avoid creating a secondary bottleneck.
2. **Low-cost:** reallocate 16 nodes to Standard, which the DOE shows compounds with weight rebalancing to a 99.2% reduction in $P_{b,\text{Standard}}$ under overload.
3. **No-cost, complementary:** enable aging-based priority promotion at $t_{\text{age}} \approx 1.5\text{--}2$ h, which independently cut blocking by $\sim 96\%$ in the preliminary sweep (Section 11).
4. **Data-driven, long-term:** instrument the cluster to capture actual vs declared walltime, replacing the assumed routing weights and log-normal service parameters with empirical distributions.

The overall conclusion is that the system as specified is not under-provisioned in aggregate, it is **mis-routed**.

11 Further Analysis / Proposed Improvements

The baseline model deliberately keeps the three partitions independent (SS_01, SS_02). The most promising extension is a **scheduler-side mitigation** that couples them: aging-based priority promotion. A job that has waited longer than a threshold t_{age} in its partition is *promoted* into a less-loaded partition, draining the saturated Standard queue without adding hardware.

11.1 Aging-Based Priority Promotion (Design)

Specifying this extension would re-introduce three hypotheses retired from the baseline:

Table 18: Additional hypotheses for aging-based priority promotion

ID	Class	Statement
(A1)	Simplifying	Priority promotion is instantaneous once t_{age} is crossed; running jobs are not preempted.
(A2)	Structural	Promotion bypasses the target partition’s admission gateway; a promoted job always enters regardless of current queue length.
(A3)	Data	Aging thresholds (e.g. $t_{\text{age}} = 1.5$ h Standard→Short, 24 h Long→Standard) are tunable control parameters.

In the DES this is a localised change: arm a `Promotion` event at `clock+tage` when a job joins the queue, and on firing relocate the job (if still waiting) into the target partition. The implementation already exists in `queue_model.py` (`_handle_promotion`, `promote_to`, `t_age`).

11.2 Preliminary Sweep (Exploratory)

A preliminary run with promotion enabled swept the Standard→Short threshold t_{age} over six values (all other parameters at baseline):

Table 19: Aging threshold sweep results

t_{age} (h)	Std W_q (h)	Short W_q (h)	Promotions	Total blocked
6.0	3.918	0.000	0	15,046
3.0	2.679	3.334	15,003	654
2.0	1.705	2.721	15,132	618
1.5	1.212	2.419	15,182	656
1.0	0.729	2.164	15,410	687
0.5	0.288	1.852	16,343	711



Figure 4: Effect of the aging threshold on waiting time and total blocking.

Three observations motivate this as future work:

1. **A high threshold is inert** — at $t_{\text{age}} = 6$ h no promotions fire; at $\rho = 0.999$ jobs complete or are blocked before the timer elapses (the row reproduces the no-aging baseline of Section 3.1).
2. **Promotion redistributes congestion** — lowering t_{age} cuts Standard W_q but loads Short (its W_q reaches 1.85 h at $t_{\text{age}} = 0.5$ h). The bottleneck shifts rather than disappearing.
3. **Total blocking is minimised around $t_{\text{age}} \approx 1.5$ –2 h** — blocking falls from 15,046 to ~ 620 ($\sim 96\%$) through scheduler configuration alone; the precise optimum within that range sits inside single-replication noise.

A RNG Validation

The **custom LCG** ($m = 2^{31}$, $a = 1103515245$, $c = 12345$) is the actual generator behind `util.exponential()` and `util.lognormal()` used throughout the Python DES (Sections 6–8); the samplers draw from `util.LCG` via inverse-CDF (exponential) and Box–Muller (log-normal) transforms. Python’s Mersenne Twister is included only as a reference comparison.

Table 20: RNG test battery

Test	What it detects	H_0
A. Frequency (KS)	Global non-uniformity over $[0, 1)$	$x \sim \text{Uniform}(0, 1)$
B. Gap	Clustering/voids — gap lengths between hits in $[0, 0.5)$	gap lengths $\sim \text{Geometric}(0.5)$
C. Order ($d = 4$)	Higher-dimensional correlation via permutation patterns of 4-tuples	all $4!$ orderings equally likely
D. Runs (up/down)	Trends/oscillation patterns	run count matches independence
E. Serial (lag-1)	Correlation between consecutive draws	$\text{corr}(x_i, x_{i+1}) = 0$

A p-value ≥ 0.05 fails to reject H_0 (“appears random”); $p < 0.05$ indicates a detected pattern.

Table 21: RNG validation results ($N = 10,000$, seed= 42)

Test	LCG stat.	LCG p	LCG	MT stat.	MT p	MT
A. Frequency	0.00665	0.765	RANDOM	0.00626	0.826	RANDOM
B. Gap	8.522	0.483	RANDOM	7.169	0.620	RANDOM
C. Order	29.389	0.168	RANDOM	25.779	0.311	RANDOM
D. Runs	-0.293	0.770	RANDOM	0.016	0.987	RANDOM
E. Serial	0.00035	0.972	RANDOM	0.00440	0.660	RANDOM

Both generators pass all five tests at $\alpha = 0.05$. The LCG results back the simulation’s validity, since `util.exponential()` and `util.lognormal()` sample from the LCG via inverse-CDF / Box–Muller transforms; the Mersenne Twister column is shown as a reference and passes equally well (comparable or higher p-values). Reproducibility: the full battery implementation is in `rng_validation.py`.

B GPSS Model Source

The validated GPSS World model is in `gpss/hpc_validation_final.gps`, and raw per-replication results are in `gpss/hpc_validation_results.csv` (Section 6.3).

```
* RUN PROCEDURE (do this for each replication):
*   1. RMULT  s1,s2,s3,s4          <- four distinct seeds, e.g. 11,22,33,44
*   2. START  1,NP                (runs to clock=500: warm-up timer fires)
*   3. SAVEVALUE SARR,0
*   4. SAVEVALUE SBLK,0
*   5. START  1,NP                (runs to clock=4500: end timer fires;
*                                collection window = 4000h)
*   6. Read off with SHOW commands (see bottom of this file)
*   7. To start the next replication: CLEAR, then go back to step 1
*      with a NEW set of seeds.
```

IAT (mean inter-arrival time) is set per scenario: 0.022222 h for $\lambda = 45$ (saturated baseline) and 0.026316 h for $\lambda = 38$ (high-subcritical cross-check, Section 6.3). RNG streams 1 (arrivals/routing) and 2–4 (Short/Standard/Long service) are reseeded via `RMULT` between the 5 replications run per scenario.